

Memory Management

Content

- LIFO
- Introduction to call stack
- Types of memory
- Quizes .

Amreshwar

Anil

ANUBHAV RAMNANI

Arunava Basak

Bhaves Pandey

Deepshikha Arora

Dhruv

Farzana kauser

Harikrishnan A

Harshad Sanjay Marathe

Iti

jeevanantham

M S Haseeb Khan

Mahesh

Manohar A N

MOHAMMAD ALI

Nutan

Pranjul Kesharwani

Priyank Varshney

Rishav Ghosh

Shivam Shiv

Shivanand Patil

Tanya

Rules

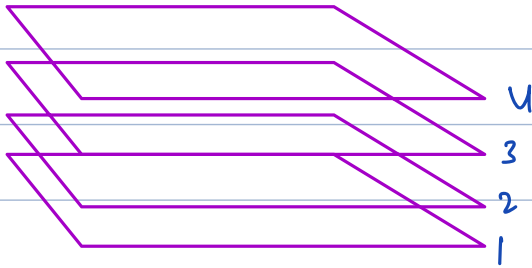
Q — QT

Ans —→ PC

Always Active

LIFO

Last in first Out.



which plate was placed last ? 4th

Last In

which plate will be removed first ? 4th

First Out

Other Examples

CD disk containers

Chairs on top of each other.

Pack of tennis balls or shuttle

Introduction to call stack

Call stack stores function call in LIFO manner.

Java

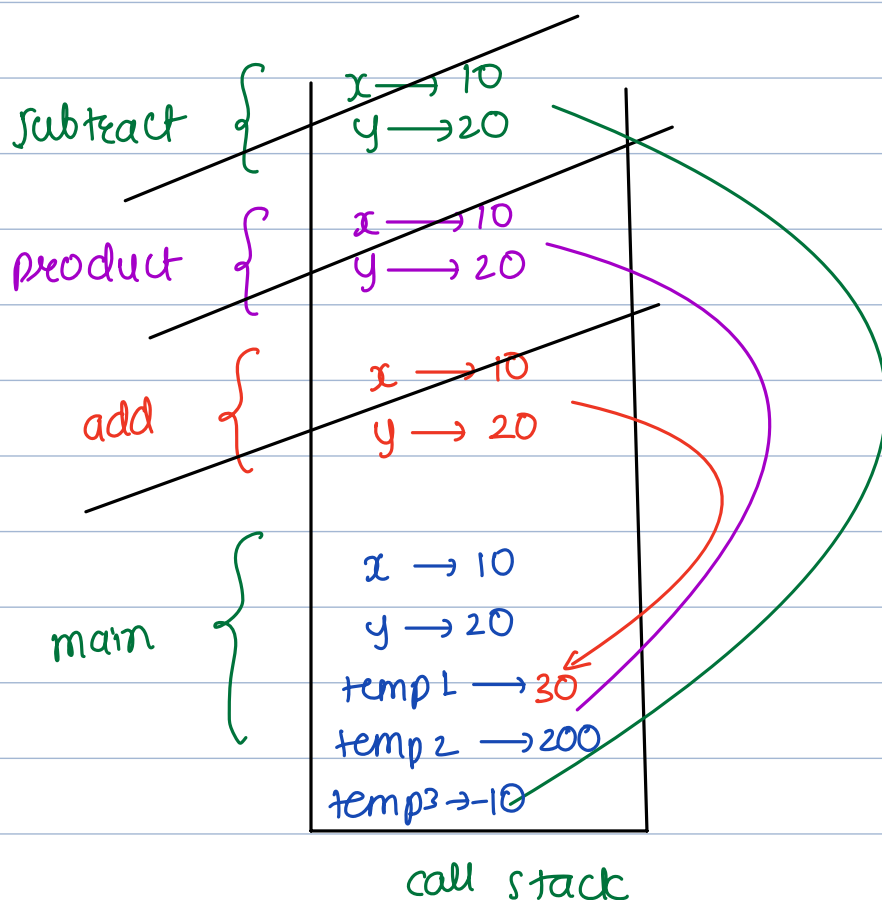
```
1 int add(int x, int y) {  
2     return x + y;  
3 }  
4 int product(int x, int y) {  
5     return x * y;  
6 }  
7 int subtract(int x, int y) {  
8     return x - y;  
9 }  
10 public static void main() {  
11     int x = 10; ✓  
12     int y = 20; ✓  
13     int temp1 = add(x, y); ✓  
14     int temp2 = product(x, y); ✓  
15     int temp3 = subtract(x, y); ✓  
16     System.out.println(temp1 + temp2 + temp3);  
17 }
```

Python

```
1 def add(x, y):  
2     return x + y  
3 def product(x, y):  
4     return x * y  
5 def subtract(x, y):  
6     return x - y  
7 def main():  
8     x = 10  
9     y = 20  
10    temp1 = add(x, y)  
11    temp2 = product(x, y)  
12    temp3 = subtract(x, y)  
13    print(temp1 + temp2 + temp3)  
14  
15 if __name__ == "__main__":  
16     main()
```

Output

220



```

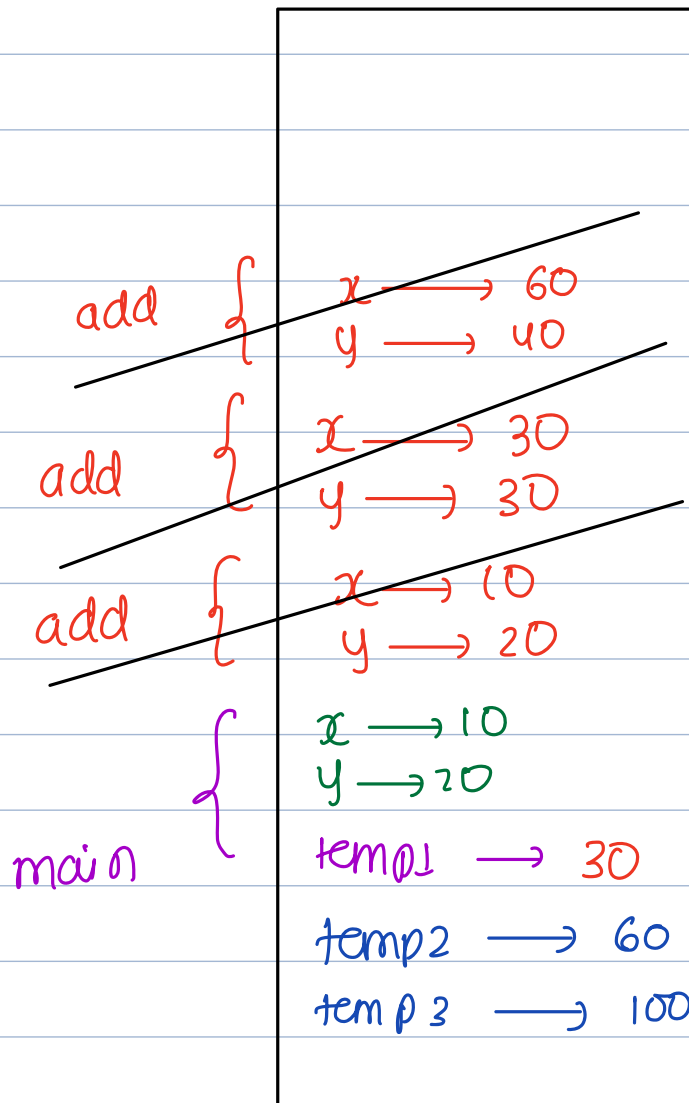
1  int add(int x, int y) {
2      return x + y;
3  }
4
5  public static void main() {
6      int x = 10;
7      int y = 20;
8      int temp1 = add(x, y);
9      int temp2 = add(temp1, 30);
10     int temp3 = add(temp2, 40);
11     System.out.println(temp3);
12 }

```

```

1  def add(x, y):
2      return x + y
3
4  def main():
5      x = 10
6      y = 20
7      temp1 = add(x, y)
8      temp2 = add(temp1, 30)
9      temp3 = add(temp2, 40)
10     print(temp3)
11
12 if __name__ == "__main__":
13     main()

```



```

1  int add(int x, int y) {
2      return x + y;
3  }
4  static int fun(int a, int b) {
5      int sum = add(a, b); ✓
6      int ans = sum * 10; ✓
7      return ans; ✓
8  }
9  static void extra(int w){
10     System.out.println("Hello"); ✓
11     System.out.println(w); ✓
12 }
13 public static void main() {
14     int x = 10; ✓
15     int y = 20; ✓
16     int z = fun(x, y); ✓
17     System.out.println(z); ✓
18     extra(z); ✓
19 }

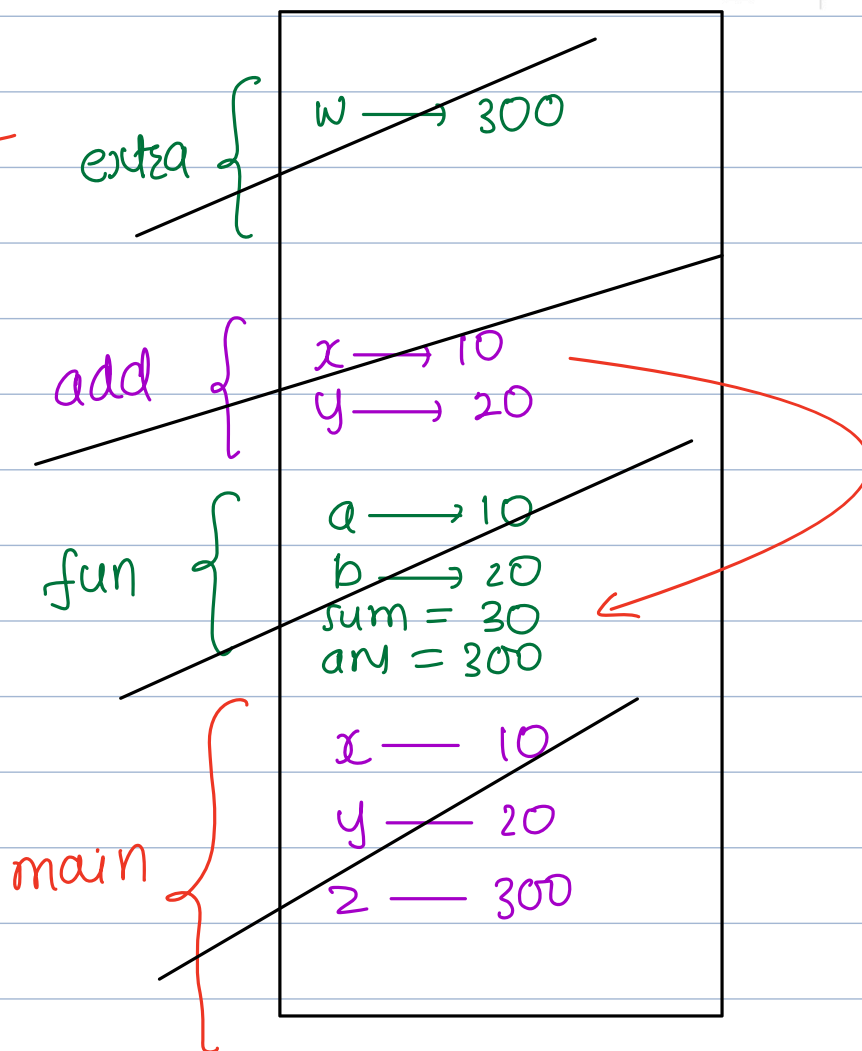
```

```

1  def add(x, y):
2      return x + y
3
4  def fun(a, b):
5      sum = add(a, b)
6      ans = sum * 10
7      return ans
8
9  def extra(w):
10     print("Hello")
11     print(w)
12
13 def main():
14     x = 10
15     y = 20
16     z = fun(x, y)
17     print(z)
18     extra(z)
19
20 if __name__ == "__main__":
21     main()

```

Output
300
Hello
300



What is reference variable ?

Address

reference
variable



hold the address of human defined container

Types of memory

Stack

Any primitive data types, reference variables, ...

Heap

The data pointed by the reference variable is stored in heap

Eg Arrays, ArrayList etc....

```
1 public static void main() {  
2     int x = 10; ✓  
3     int[] ar = new int[3]; ✓  
4     System.out.println(ar); ✓  
5     System.out.println(ar[2]); ✓  
6     ar[1] = 7; ✓  
7 }
```

Output → 2k
0

```
1 def main():  
2     x = 10  
3     ar = [0] * 3  
4     print(ar)  
5     print(ar[2])  
6     ar[1] = 7  
7  
8 if __name__ == "__main__":  
9     main()
```

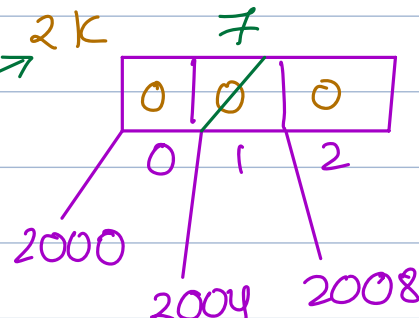
Stack

main {

x → 10

ar → 2k

Heap



$2k + 4 \times 2$

2008

primitive data types

{ stack[✓] or Heap }

Reference / address

{ stack[✓] or Heap }

Container / Array / ArrayList

{ stack or Heap[✓] }

```

1 public static void main() {
2     int x = 10; ✓
3     int[] ar = new int[3]; ✓
4     int[] ar2 = ar;
5     System.out.println(ar);
6     System.out.println(ar2);
7 }

```

Output ——— 2k
2k

Stack

main {

x — 10

ar — 2k

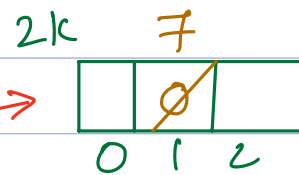
ar2 — 2k

```

1 def main():
2     x = 10
3     ar = [0] * 3
4     ar2 = ar
5     print(ar)
6     print(ar2)
7
8 if __name__ == "__main__":
9     main()

```

Heap



```

1 static void fun(int[] a){
2     System.out.println(a);
3     a[1] = 5;
4 }
5 public static void main() {
6     int[] ar = new int[3]; ✓
7     System.out.println(ar); ✓
8     ar[0] = 90; ✓
9     ar[1] = 50; ✓
10    fun(ar); ✓
11    System.out.println(ar[1]);
12 }

```

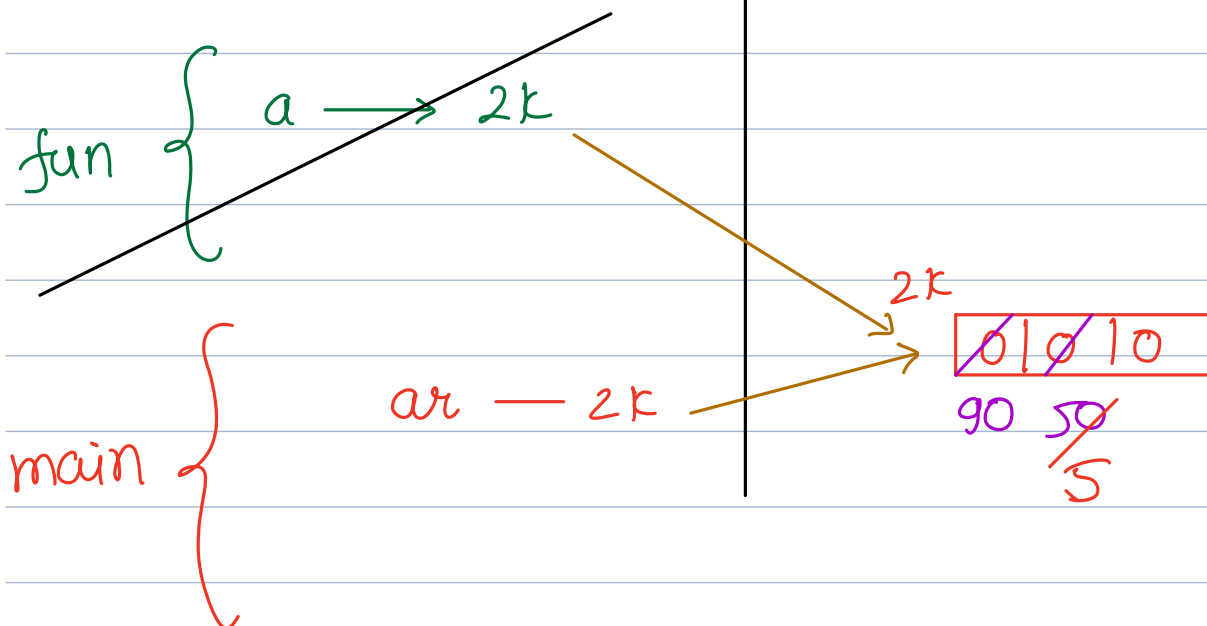
```

1 def fun(a):
2     print(a)
3     a[1] = 5
4
5 def main():
6     ar = [0] * 3
7     print(ar)
8     ar[0] = 90
9     ar[1] = 50
10    fun(ar)
11    print(ar[1])
12
13 if __name__ == "__main__":
14     main()

```

Stack

Heap



Output

2k

5

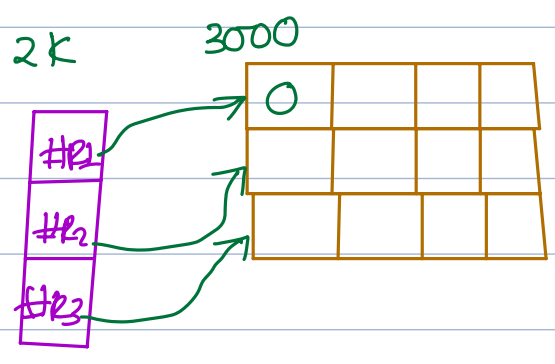
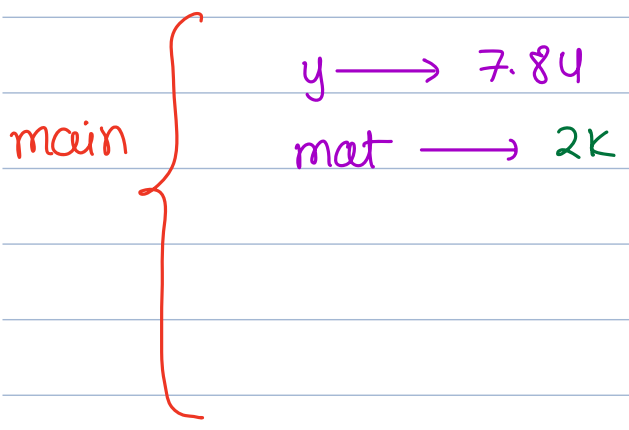
```
1 public static void main() {
2     float y = 7.84f; ✓
3     int[][] mat = new int[3][4]; ✓
4     System.out.println(mat); ✓
5     System.out.println(mat[1]); ✓
6     System.out.println(mat[1][3]);
7 }
```

```
1 def main():
2     y = 7.84
3     mat = [[0]*4 for _ in range(3)]
4     print(mat)
5     print(mat[1])
6     print(mat[1][3])
7
8 if __name__ == "__main__":
9     main()
```

Stack

Heap

#P1 = 3K



Output — 2k

2016

0

```

1 static void sum(int[][] mat){
2     System.out.println(mat);
3     System.out.println(mat[0][0] + mat[1][0]);
4 }
5 public static void main() {
6     int[][] mat = new int[2][3]; ✓
7     mat[0][0] = 15; ✓
8     mat[1][0] = 25; ✓
9     sum(mat); ✓
10 }

```

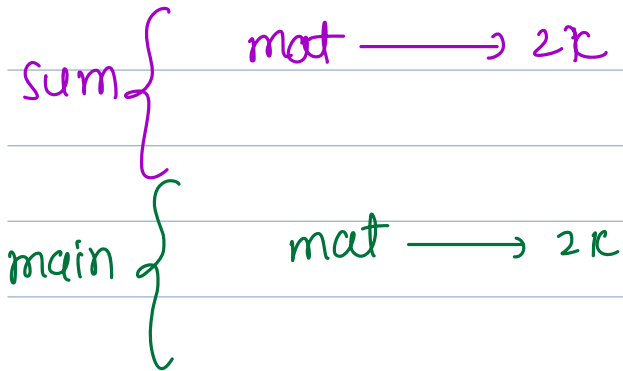
output → 2k
40

```

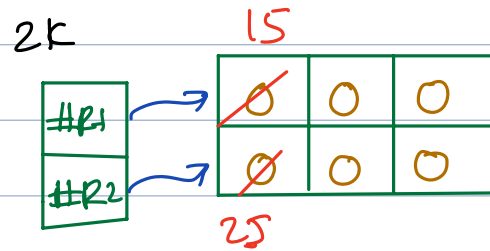
1 def sum(mat):
2     print(mat)
3     print(mat[0][0] + mat[1][0])
4
5 def main():
6     mat = [[0] * 3 for _ in range(2)]
7     mat[0][0] = 15
8     mat[1][0] = 25
9     sum(mat)
10
11 if __name__ == "__main__":
12     main()

```

Stack



Heap



```

1 static int sumOfRow(int[] arr){
2     System.out.println(arr); ✓
3     int sum = 0; ✓
4     for (int i = 0; i < arr.length; i++){
5         sum = sum + arr[i]; ✓
6     }
7     return sum; ✓
8 }
9
10 public static void main() {
11     int[][] mat = new int[2][3]; ✓
12     mat[0][0] = 9; ✓
13     mat[0][1] = 5; ✓
14     mat[0][2] = 1; ✓
15     int ans = sumOfRow(mat[0]); ✓
16     System.out.println(ans);
17 }

```

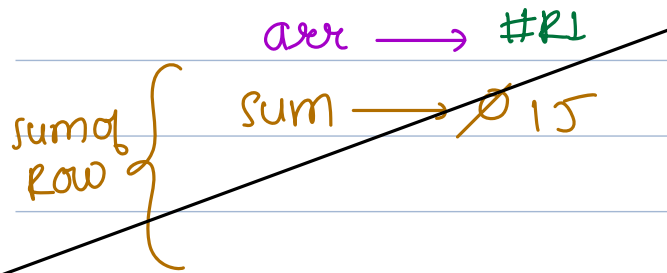
output — ~~#1~~ 15

```

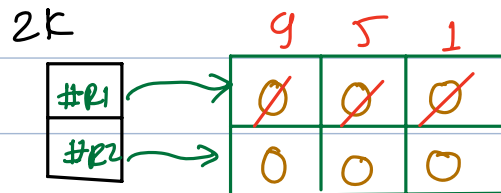
1 def sum_of_row(arr):
2     print(arr)
3     total = 0
4     for num in arr:
5         total += num
6     return total
7
8 def main():
9     mat = [[0] * 3 for _ in range(2)]
10    mat[0][0] = 9
11    mat[0][1] = 5
12    mat[0][2] = 1
13    ans = sum_of_row(mat[0])
14    print(ans)
15
16 if __name__ == "__main__":
17     main()

```

Stack



Heap



Break : 22 : 34

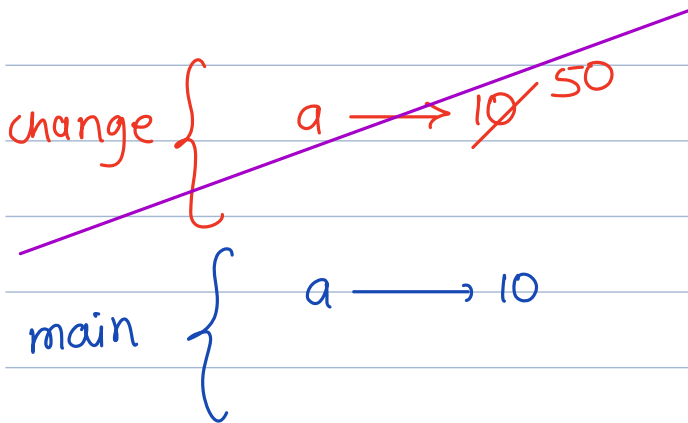
Java

```
static void change(int a) {  
    a = 50; ✓  
}  
  
public static void main(String args[]) {  
    int a = 10; ✓  
    change(a); ✓  
    System.out.println(a); 10  
}
```

Python

```
def change(a):  
    a = 50  
  
def main():  
    a = 10  
    change(a)  
    print(a)  
  
if __name__ == "__main__":  
    main()
```

stack



Heap

Java

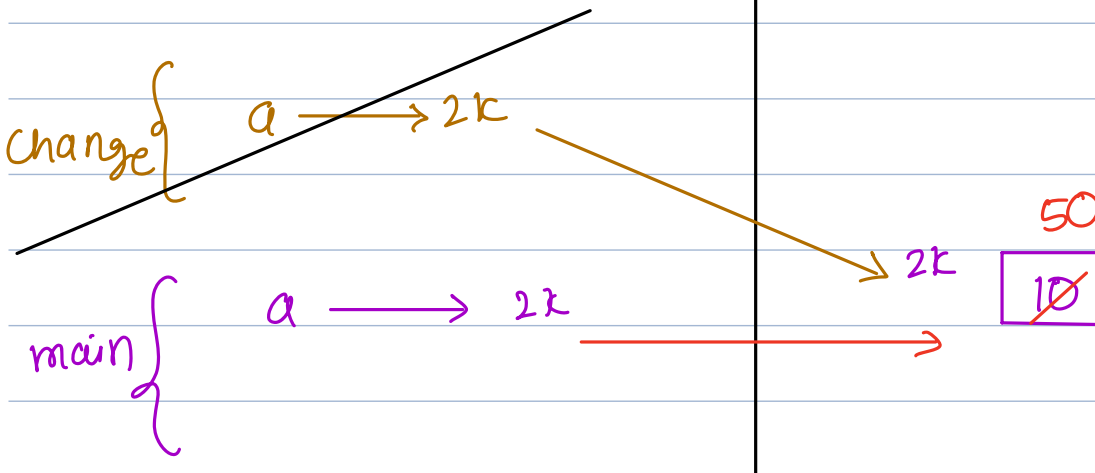
```
static void change(int[]a) {  
    a[0] = 50;  
}  
  
public static void main(String args[]) {  
    int[]a = {10}; ✓  
    change(a);  
    System.out.println(a[0]); 50  
}
```

Python

```
def change(a):  
    a[0] = 50  
  
def main():  
    a = [10]  
    change(a)  
    print(a[0])  
  
if __name__ == "__main__":  
    main()
```

stack

heap

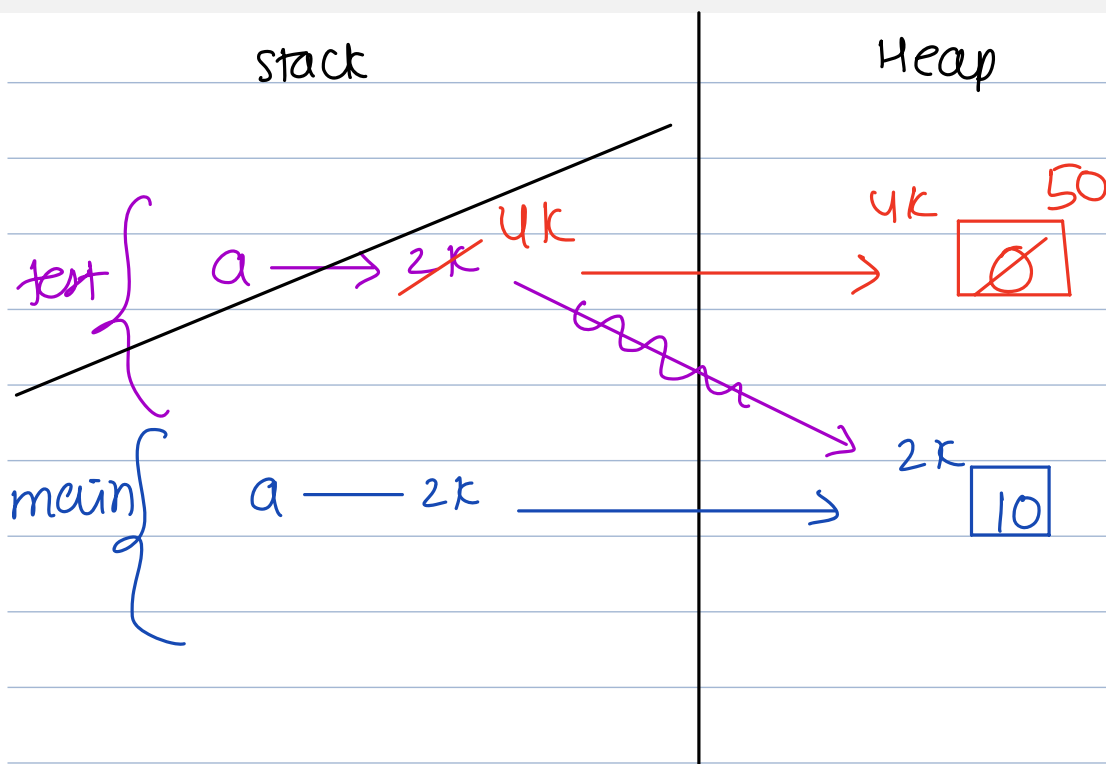


Java

```
static void test(int[] a) {  
    a = new int[1]; ✓  
    a[0] = 50; ✓  
}  
  
public static void main(String args[]) {  
    int[] a = {10}; ✓  
    test(a); ✓  
    System.out.println(a[0]); // 10  
}
```

Python

```
def test(a):  
    a = [0]  
    a[0] = 50  
  
def main():  
    a = [10]  
    test(a)  
    print(a[0])  
  
if __name__ == "__main__":  
    main()
```



Java

```
static void fun(int[] a) {  
    a = new int[1];  
    a[0] = 100;  
}  
  
public static void main() {  
    int[] a = {10, 20, 30};  
    fun(a);  
    System.out.println(a[0]);  
}
```

Python

```
def fun(a):  
    a = [0]  
    a[0] = 100  
  
def main():  
    a = [10, 20, 30]  
    fun(a)  
    print(a[0])  
  
if __name__ == "__main__":  
    main()
```

stack

Heap

Java

```
static void swap(int a,int b) {  
    int temp = a;  
    a = b;  
    b = temp;  
}  
  
public static void main(String args[]) {  
    int a = 10;  
    int b = 20;  
    swap(a,b);  
    System.out.println(a + " " + b);  
}
```

Python

```
def swap(a, b):  
    temp = a  
    a = b  
    b = temp  
  
def main():  
    a = 10  
    b = 20  
    swap(a, b)  
    print(a, b)  
  
if __name__ == "__main__":  
    main()
```

Java

```
static void swap(int[]a,int[]b) {  
    int temp = a[0];  
    a[0] = b[0];  
    b[0] = temp;  
}  
  
public static void main(String args[]) {  
    int[]a = {10};  
    int[]b = {20};  
    swap(a,b);  
    System.out.println(a[0] + " " + b[0]);  
}
```

Python

```
def swap(a, b):  
    temp = a[0]  
    a[0] = b[0]  
    b[0] = temp  
  
def main():  
    a = [10]  
    b = [20]  
    swap(a, b)  
    print(a[0], b[0])  
  
if __name__ == "__main__":  
    main()
```

Java

```
static int[] fun(int[] a) {  
    a = new int[2];  
    a[0] = 50; a[1] = 60;  
    return a;  
}  
  
public static void main(String args[]) {  
    int[] a = {10, 20, 30};  
    a = fun(a);  
    System.out.println(a[0]);  
}
```

Python

```
def fun(a):  
    a = [0, 0]  
    a[0] = 50  
    a[1] = 60  
    return a  
  
def main():  
    a = [10, 20, 30]  
    a = fun(a)  
    print(a[0])  
  
if __name__ == "__main__":  
    main()
```

Java

```
static void test(int[]a) {  
    a = new int[2];  
    a[0] = 94;  
}  
  
public static void main(String args[]) {  
    int[]a = {10,20,30};  
    test(a);  
    System.out.println(a[0]);  
}
```

Python

```
def test(a):  
    a = [0, 0]  
    a[0] = 94  
  
def main():  
    a = [10, 20, 30]  
    test(a)  
    print(a[0])  
  
if __name__ == "__main__":  
    main()
```